

Informatica 2.2 / Computational Intelligence

E8: Swarm Intelligence

Marco S. Nobile, Ph.D.

Bachelor's Degree in Engineering Physics

Ca' Foscari University of Venice

A.Y. 2025-2026



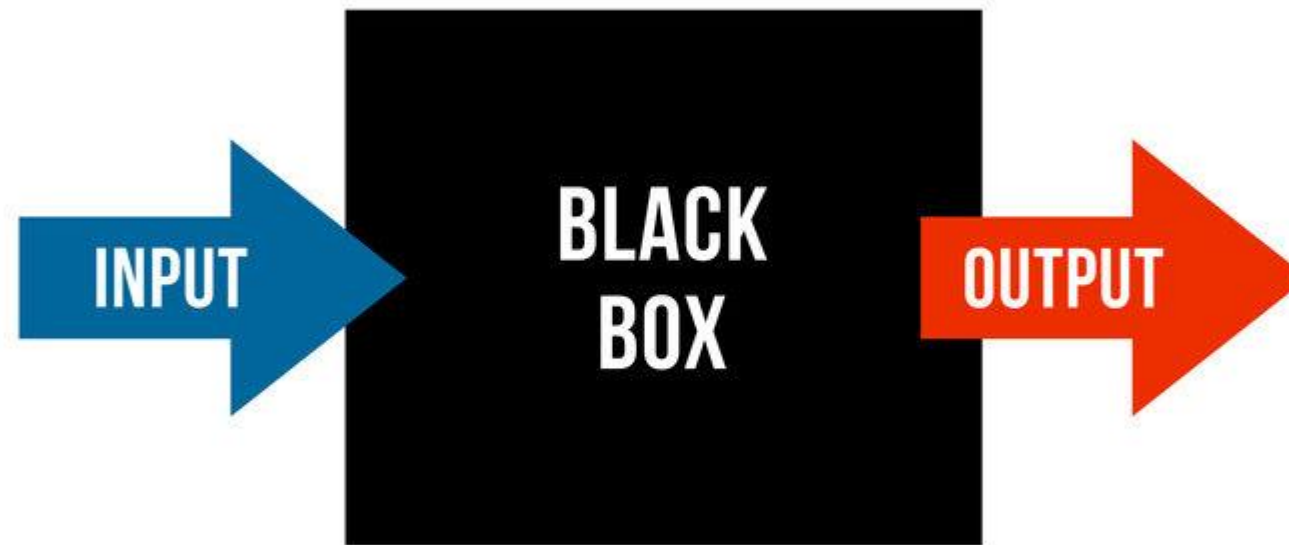
Università
Ca' Foscari
Venezia

Outline

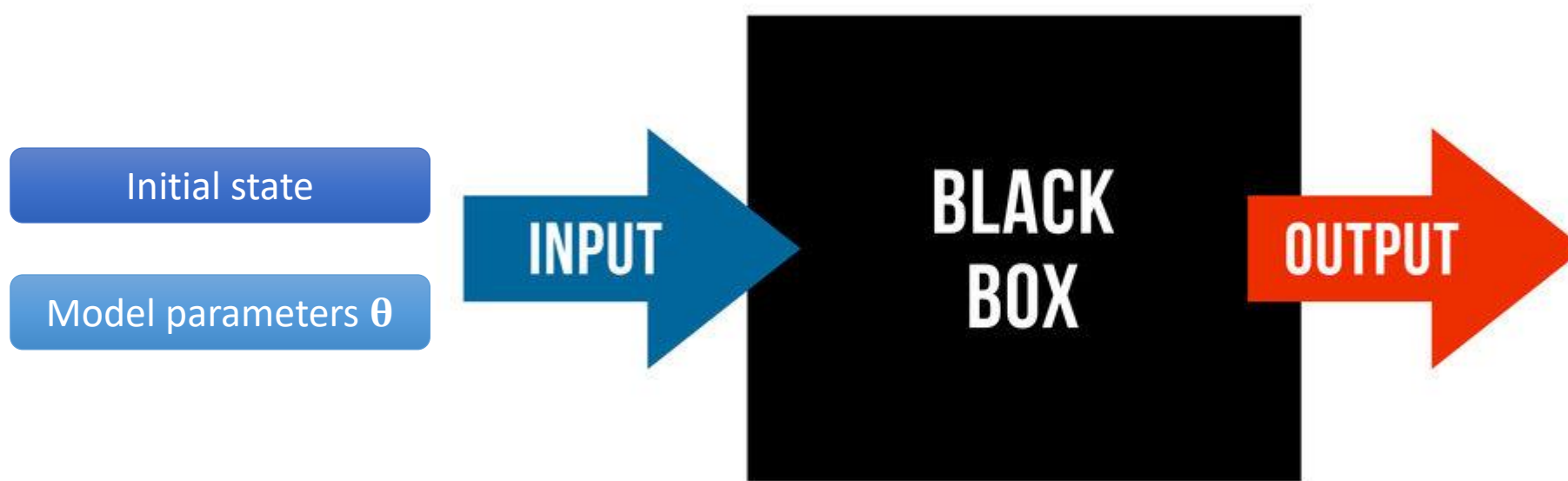


- Optimization problems
- Calibrating models as optimization problems
- Superorganisms and swarm intelligence
- Particle Swarm Optimization (PSO)
- FST-PSO
- Animal Colonies: Ants and Bees

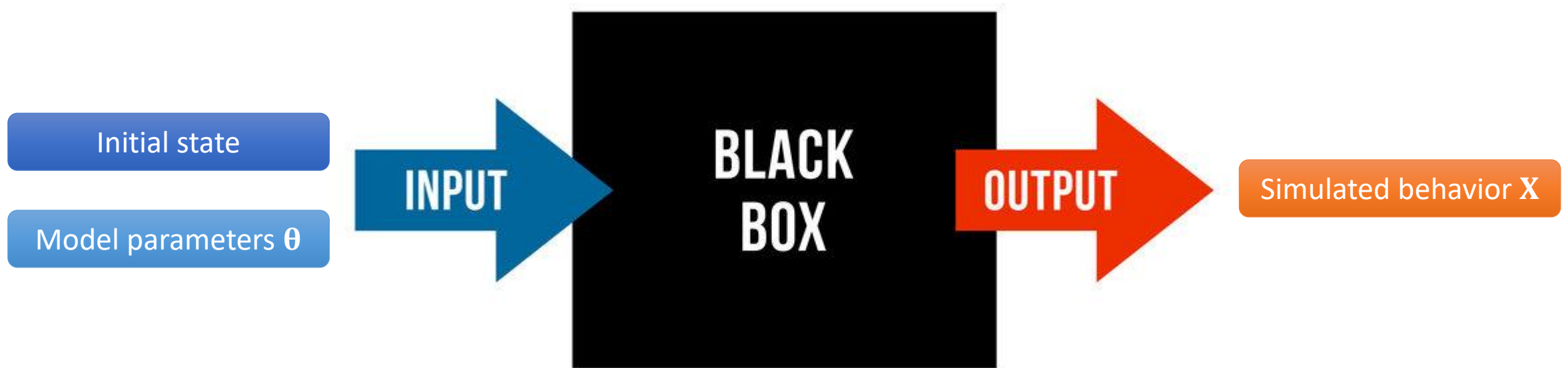
Models: an abstract view



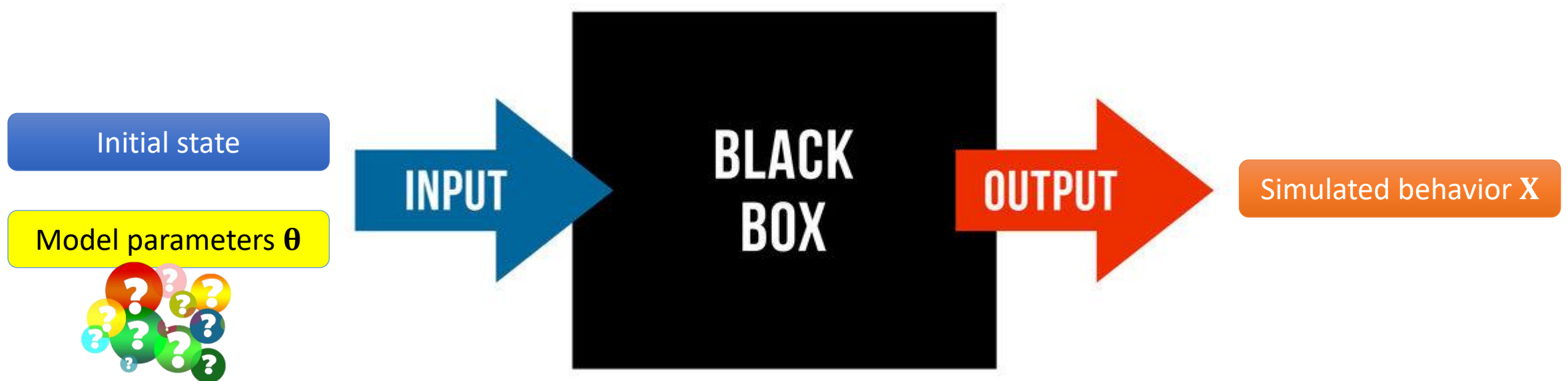
Models: an abstract view



Models: an abstract view



Models: an abstract view



The importance of parameters



Brusselator - Initial amounts:

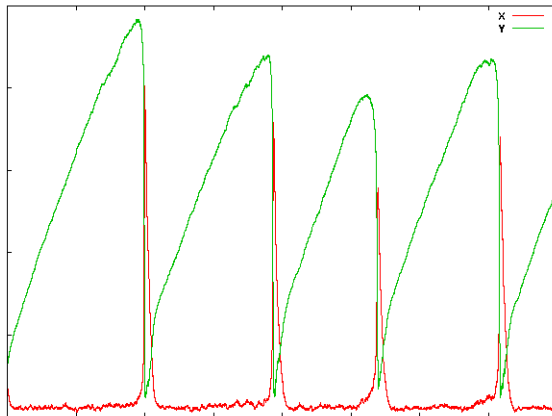
A = 200 molecules (fixed), X = 200 molecules, B = 600 molecules (fixed), Y = 300 molecules

The importance of parameters



Brusselator - Initial amounts:

A = 200 molecules (fixed), X = 200 molecules, B = 600 molecules (fixed), Y = 300 molecules



$$c_1 = 1$$

$$c_2 = 5 \times 10^{-3}$$

$$c_3 = 2.5 \times 10^{-5}$$

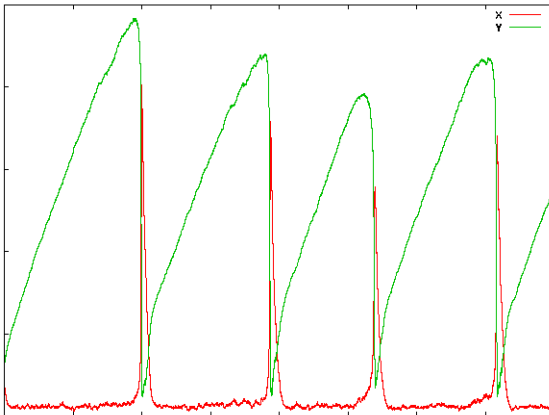
$$c_4 = 1.5$$

The importance of parameters

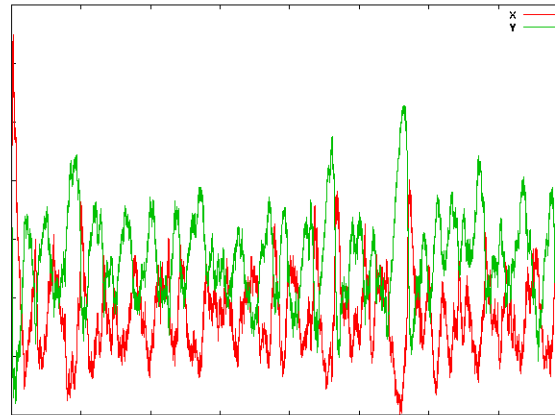


Brusselator - Initial amounts:

A = 200 molecules (fixed), X = 200 molecules, B = 600 molecules (fixed), Y = 300 molecules



$$\begin{aligned}c_1 &= 1 \\c_2 &= 5 \times 10^{-3} \\c_3 &= 2.5 \times 10^{-5} \\c_4 &= 1.5\end{aligned}$$



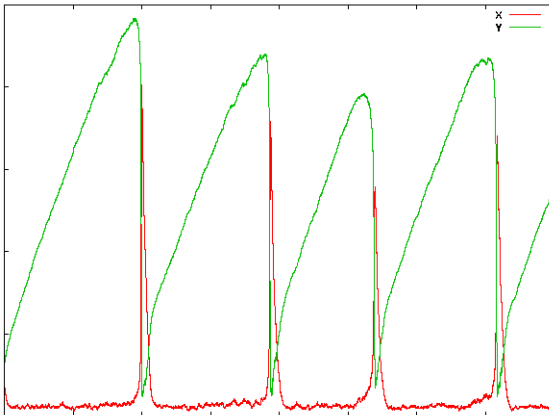
$$\begin{aligned}c_1 &= 1 \\c_2 &= 5 \times 10^{-3} \\c_3 &= 2.5 \times 10^{-4} \\c_4 &= 1.5\end{aligned}$$

The importance of parameters

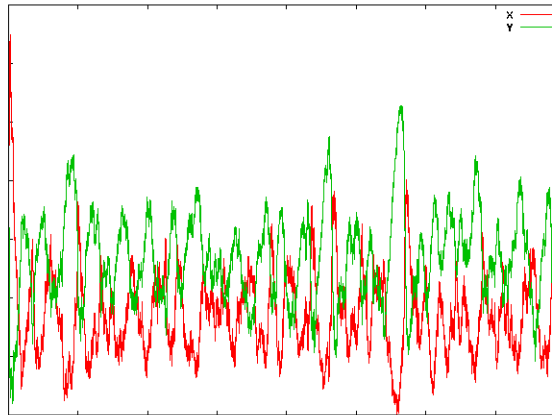


Brusselator - Initial amounts:

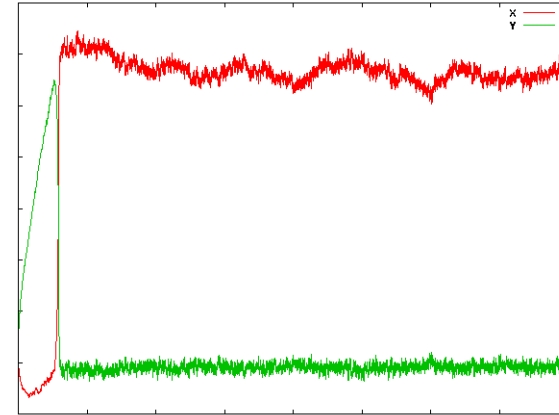
A = 200 molecules (fixed), X = 200 molecules, B = 600 molecules (fixed), Y = 300 molecules



$$\begin{aligned}c_1 &= 1 \\c_2 &= 5 \times 10^{-3} \\c_3 &= 2.5 \times 10^{-5} \\c_4 &= 1.5\end{aligned}$$



$$\begin{aligned}c_1 &= 1 \\c_2 &= 5 \times 10^{-3} \\c_3 &= 2.5 \times 10^{-4} \\c_4 &= 1.5\end{aligned}$$



$$\begin{aligned}c_1 &= 1 \\c_2 &= 5 \times 10^{-3} \\c_3 &= 2.5 \times 10^{-5} \\c_4 &= 1.5 \times 10^{-1}\end{aligned}$$



How do we fill the gap?

- We cannot measure the kinetic parameters but **we can perform experiments** and collect some «indirect» data
 - E.g., **time-series** of molecular amounts, **cells fluorescence** histograms...
 - We «**look for**» the **parameters θ** able to reproduce (*in silico*) the experimental data
 - This problem is known as **Parameter Estimation** (PE) [Moles *et al.*, Genome Res 2009]
- **Important 1**: in the PE problem, the structure of the **model is known**. If not so the problem is *reverse engineering / model inference* that is not discussed here
- **Important 2**: the **initial state of the system** can be a parameter of the model as well (possibly, unknown). The problem of estimating the initial state is not discussed in this lecture

How would YOU find the parameters?



?



A possible algorithm for PE

1. Perform a laboratory experiment and **collect some experimental data $\mathbf{Y}$**
 2. Choose a **random vector** of kinetic parameters $\boldsymbol{\theta} \in \mathbb{R}^{|R|}$
 3. Use $\boldsymbol{\theta}$ to perform a **simulation** (e.g., using differential equations or SSA)
 4. The simulated dynamics $\mathbf{X}$ is compared to the target data $\mathbf{Y}$
 5. If $\mathbf{X}$ is «different» from $\mathbf{Y}$ we **improve $\boldsymbol{\theta}$** and iterate from step #3
 6. The parameterization $\boldsymbol{\theta}$ is returned as the solution
- How can we perform steps #4 and #5 (but also #2)?

Comparing the dynamics: the *fitness function*



The **fitness function** $f(\boldsymbol{\theta})$: $\mathbb{R}^{|R|} \rightarrow \mathbb{R}^+$ performs the comparison of the **simulated dynamics** $\mathbf{X}^\theta$ and the **target data** $\mathbf{Y}$

- Given a parameterization $\boldsymbol{\theta}$ the fitness «**quantifies**» the **quality of the fit**
- It is generally defined as a sum of some **difference** between **target** and **simulation**

$$f(\boldsymbol{\theta}) = \|\mathbf{X}^\theta - \mathbf{Y}\|$$

- Note that this definition is absolutely general
 - The simulation and the target can be anything: time-series, histograms of cells' fluorescence, etc.
 - The simulation can be stochastic or deterministic
 - **Any difference measure can be used** as fitness!
 - Depending on the problem and the nature of data, we can use root mean squared error (RMSE), mean absolute percentage error (MAPE), Kullback-Leibler divergence, Hellinger distance, binary cross-entropy, etc. (more about this later)

Parameter estimation as an optimization problem

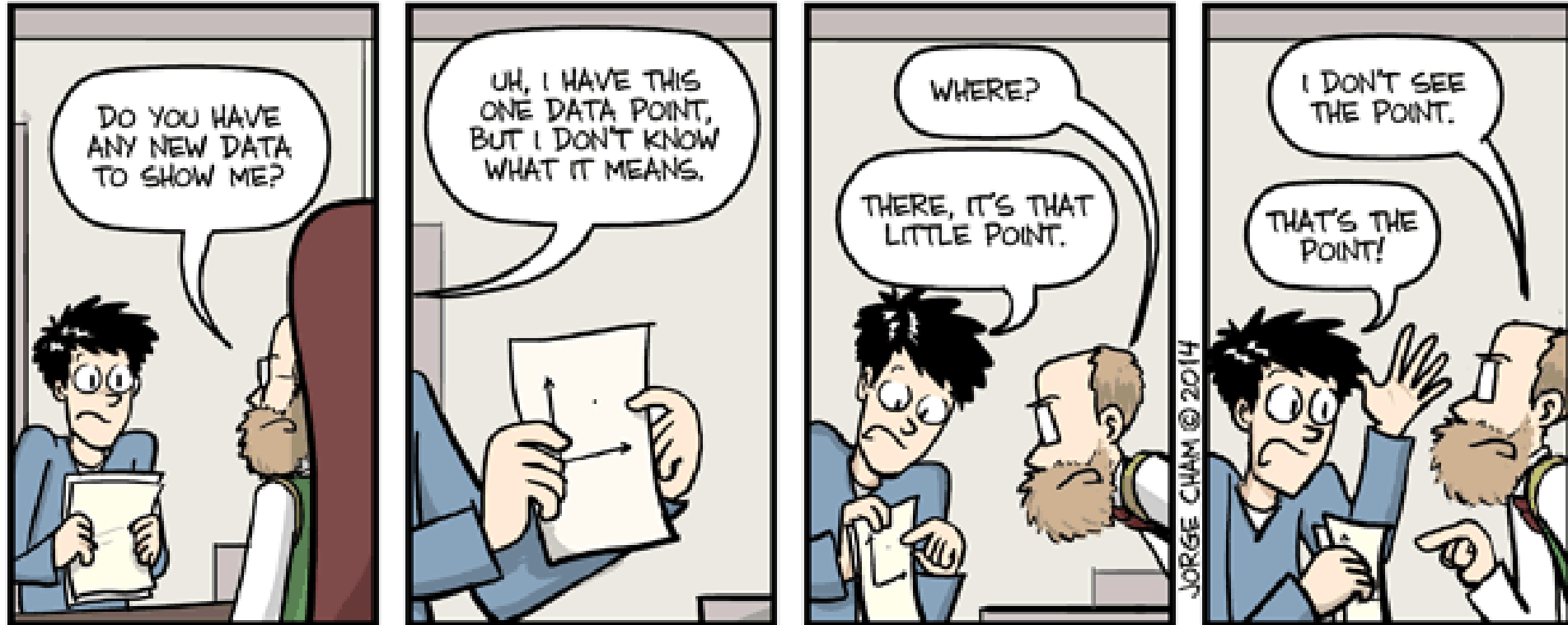


The PE problem corresponds to the problem of determining an **optimal solution $\mathbf{o}$** – in the space $\mathcal{S}$ of all possible solutions (i.e., kinetic parameterizations) – such that

$$f(\mathbf{o}) \leq f(\boldsymbol{\theta}) \quad \forall \boldsymbol{\theta} \in \mathcal{S}$$

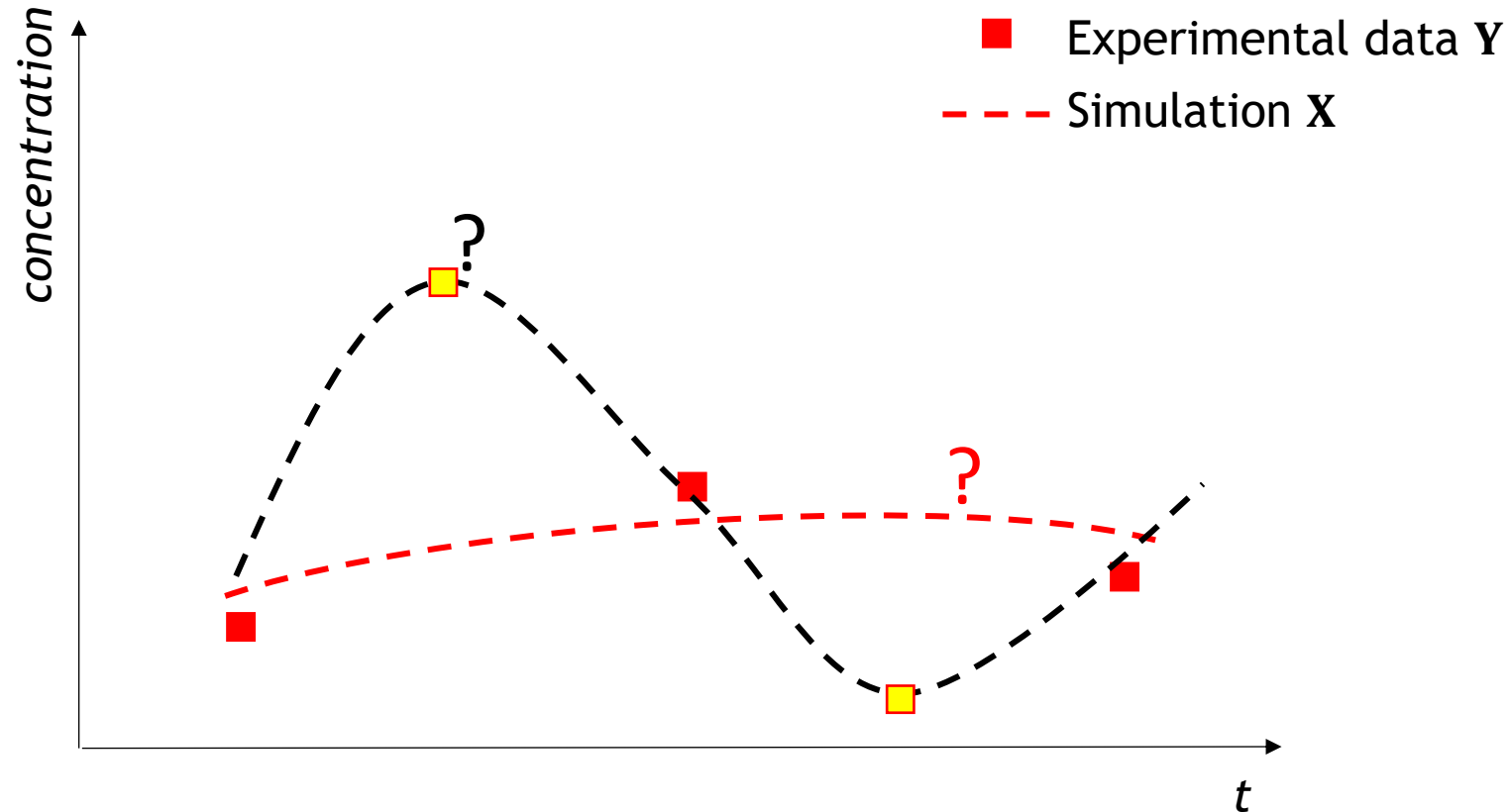
- That is, a solution that **cannot be further improved** with respect to the fitness
- The PE problem is restated as an optimization (specifically **minimization**) problem: the goal is to **minimize the distance between the target and the simulated dynamics**

A few words about experimental data



WWW.PHDCOMICS.COM

Problems of undersampling



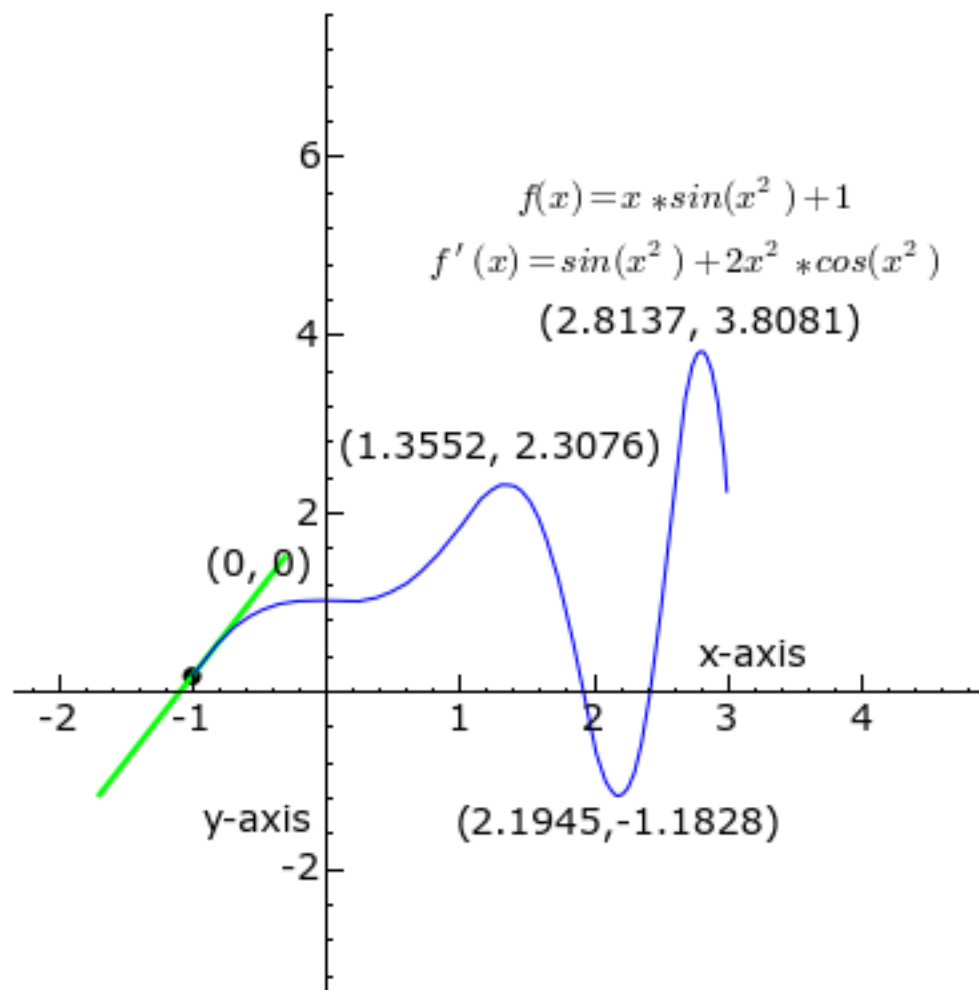
Many samples help the estimation: they allow a more faithful and **constrained** fit of the dynamics

Derivatives

Positive

Negative

Equal to 0

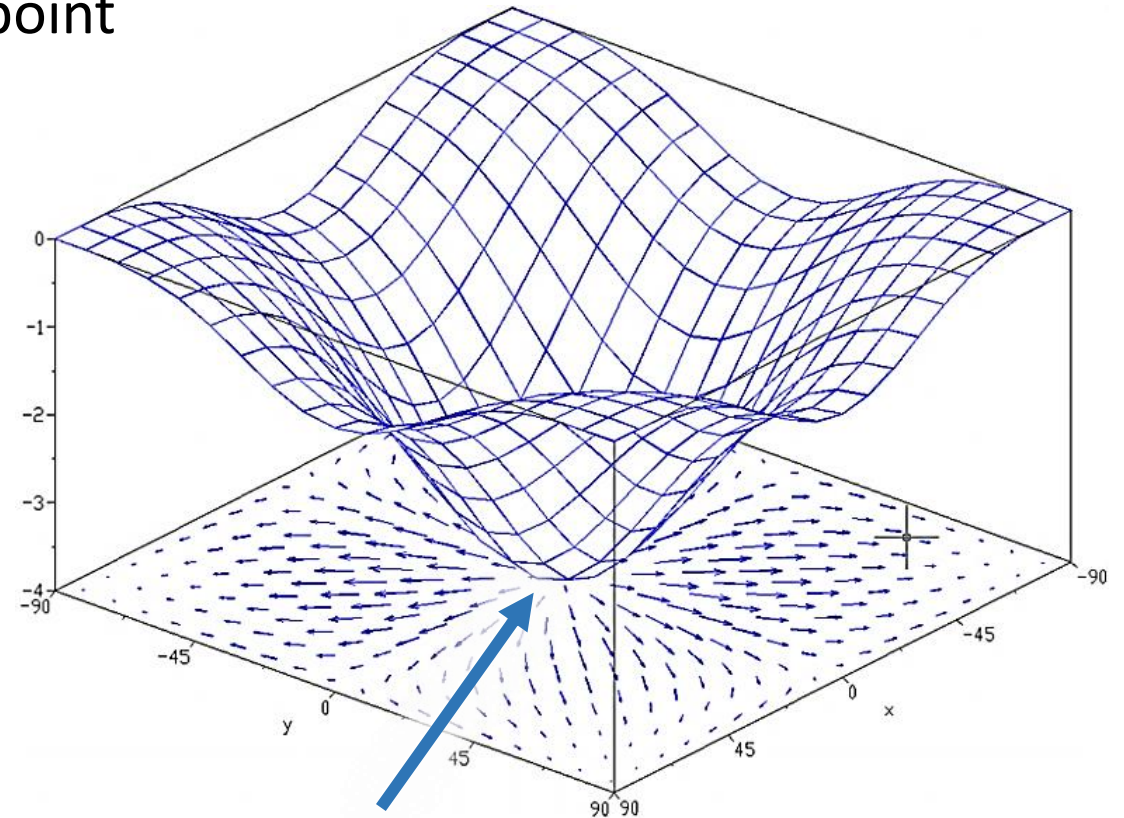


Gradient in 2D

The **gradient** of a (fitness) function in one point is a **vector** that points in the direction of the **steepest ascent**

The gradient defines a **vector field** (see the arrows on the xy plane)

$$\nabla f(x, y) = \frac{\partial f}{\partial x} \mathbf{i} + \frac{\partial f}{\partial y} \mathbf{j}$$



The gradient descent algorithm



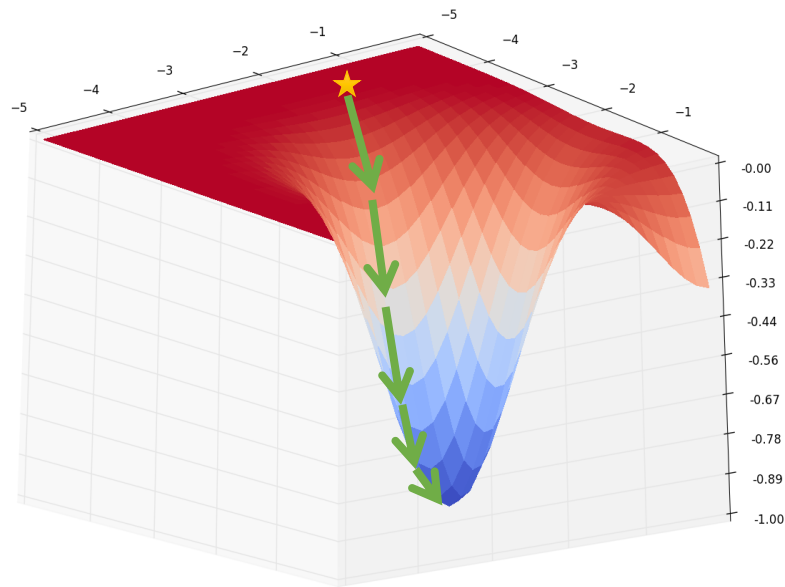
Pie courtesy of Claudio di Stefano

We can pick a **random initial solution** in the space $\mathcal{S}$ and **calculate the gradient**

- Remember that the gradient points the direction of the greatest rate of **increment** of a function
- We can take «steps» in the **opposite direction**, towards the minimum
- We iterate until the **gradient is zero** (i.e., there is no direction which improves the fitness function further)
- This algorithm is known as «**gradient descent**» (GD)
- **Levenberg-Marquardt** algorithm uses an **adaptive** step length [Marquardt, SIAM J Appl Math, 1963]
- The surface defined by the fitness function is named **fitness landscape**
- GD algorithm is basically «walking» down the fitness landscape

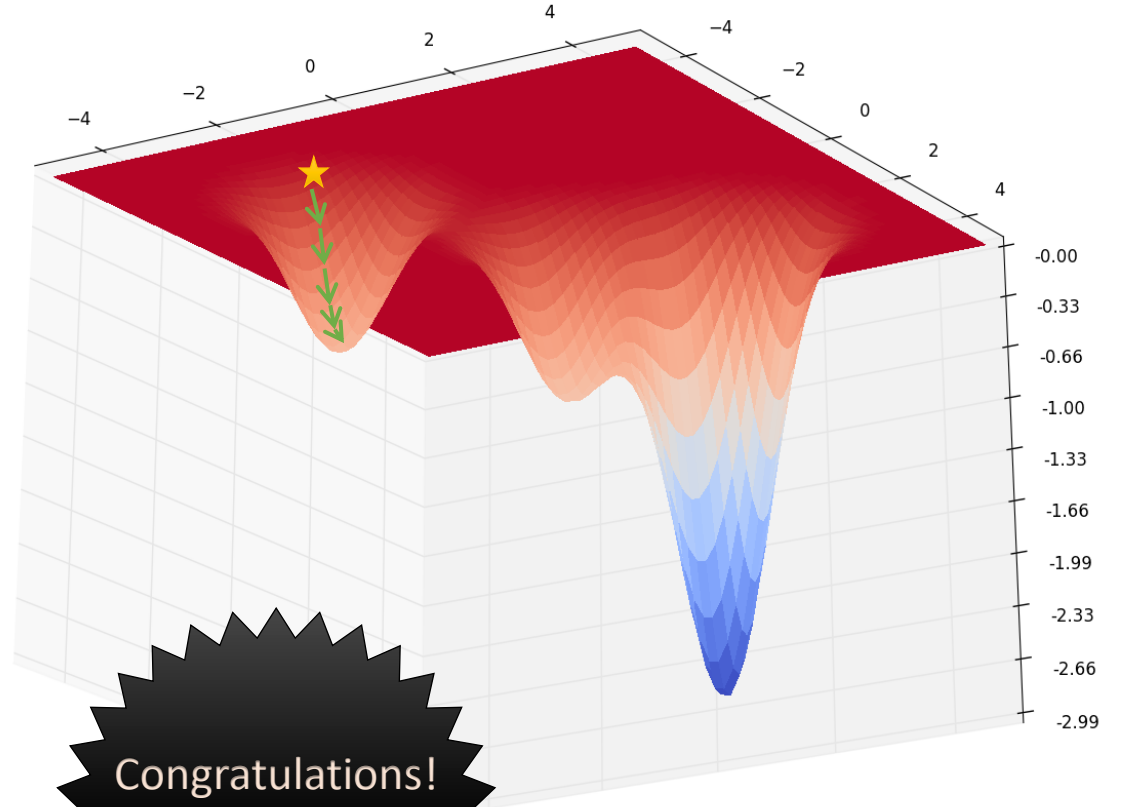


Example of execution of GD



Worst fitness Best fitness

★ Initial random solution



Congratulations!
You just fell into a
local minimum!

What now?

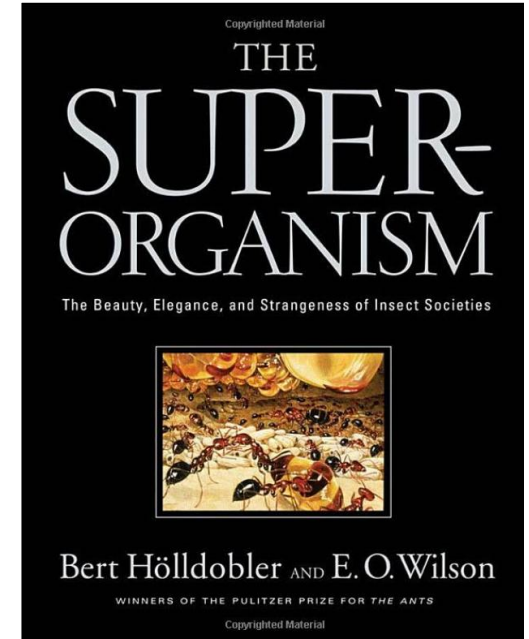


- There are more advanced methods based on this idea (e.g., multi-start GD, stochastic GD, Simulated Annealing, etc.) but **they all face the same issue**
- We need to **abandon the idea** of exploring the search space by **randomly guessing and progressively improving** a solution
- Yeah, how?

"Superorganisms"



- Swarm Intelligence is a class of **bio-inspired** algorithms that exploit the **emerging intelligence** of groups of animals
- *“Tightly knit colony of individuals, formed by altruistic cooperation, complex communication, and division of labor, which represent one of the basic stages of biological organization, midway between the organism and the entire species”*
- **Individuals are simple agents** with limited “intelligence”, limited capabilities, limited memory, limited possible behaviors
- Examples in nature: ant vs ant colony, bee vs bee colony, fish vs fish school, bird vs bird flock...

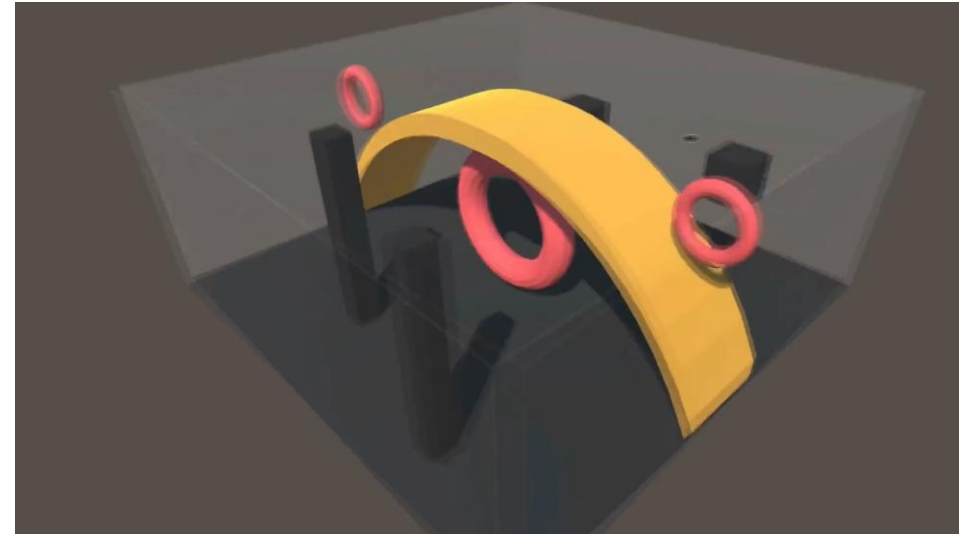
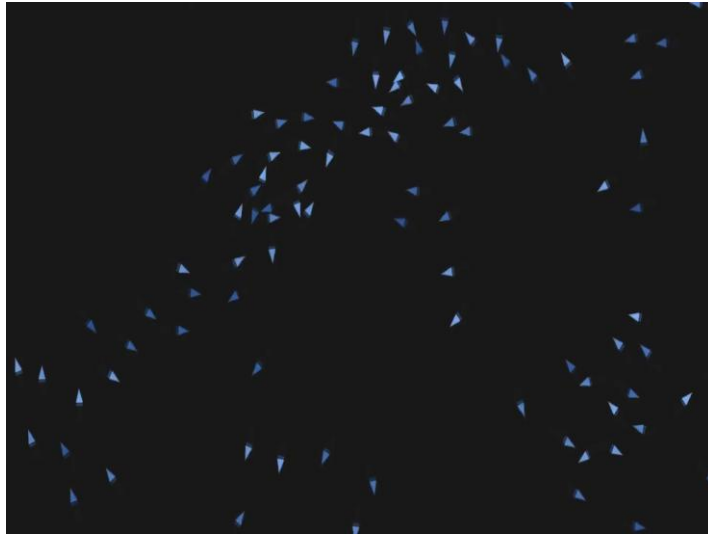


Swarm Intelligence

- Foundation: the **emergent intelligence of groups of agents**
- The individuals **spontaneously converge** to optimal solution of a complex (optimization) problem
- We will see three algorithms: **particle swarm optimization**, **ant colony optimization** and (if time permits) **artificial bee colony**
- All these algorithms are based on **information exchange** within the superorganisms about promising solutions
- Specifically, inter-swarm communication, pheromones traces, and the so-called waggle dance
- See [Eberhart et al., Swarm Intelligence, 2001]



Remember Boids?



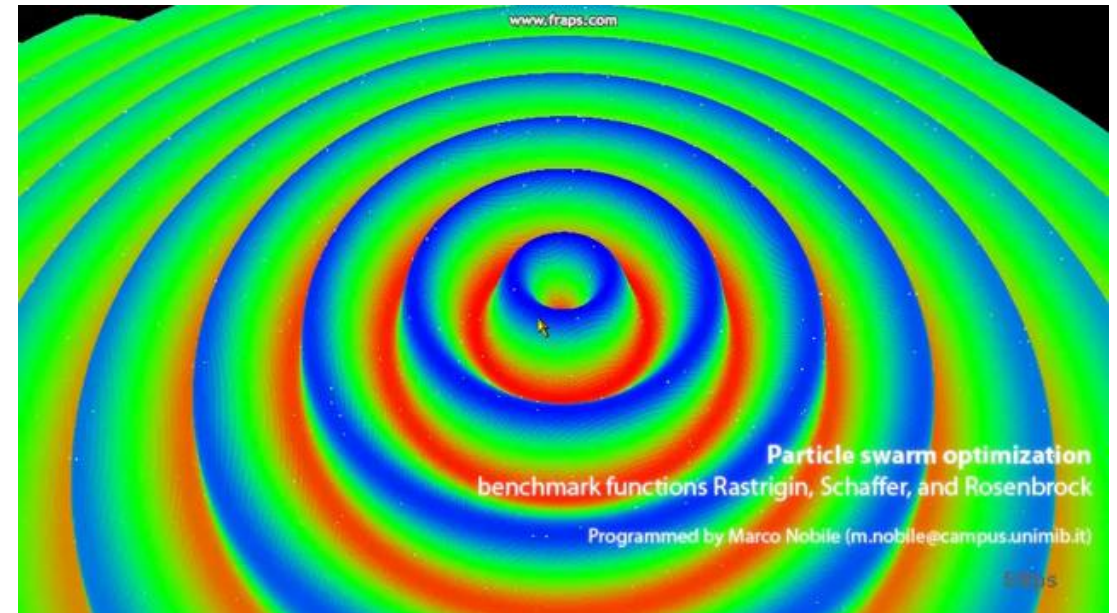
Videos courtesy of Sebastian Lague, "Coding Adventures: Boids"

<https://www.youtube.com/watch?v=bqtqltqcQhw>

Particle Swarm Optimization [1/3]



- Particle Swarm Optimization (PSO) is a Swarm Intelligence optimization meta-heuristic
 - Similarly to boids, inspired by the **collective movement of animals**
 - PSO is based on a population (the **swarm**) of N individuals (the **particles**)
 - [Kennedy and Eberhart, Proc IEEE Int Conf Neural Networks 1995]



Particle Swarm Optimization [2/3]



Particles represent candidate solutions

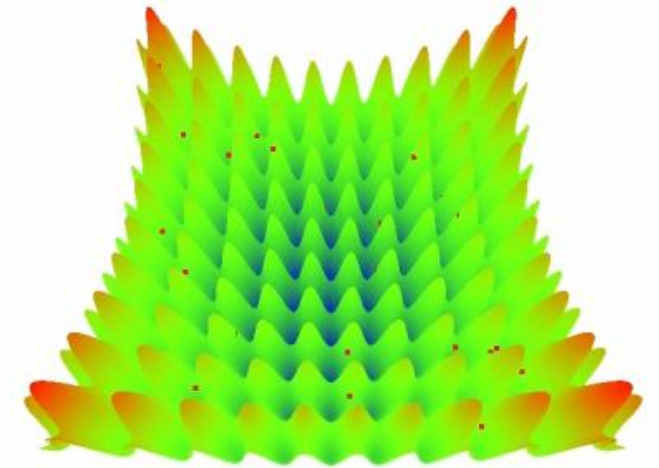
- **They move** inside a M -dimensional bounded search space
- **Particles cooperate** to identify the optimal solution

The i -th particle is characterized by **two vectors**:

- $\mathbf{x}_i(t) \in \mathbb{R}^M$ is the **position** of i -th particle in the search space during iteration t
- $\mathbf{v}_i(t) \in \mathbb{R}^M$ is the **velocity** of i -th particle during iteration t

Velocities are used to **update** particle's positions:

- $\mathbf{x}_i(t + 1) = \mathbf{x}_i(t) + \mathbf{v}_i(t)$
- Velocities change as a result of **two attractions**: social and cognitive



PSO optimizing a 2D Rastrigin's benchmark function

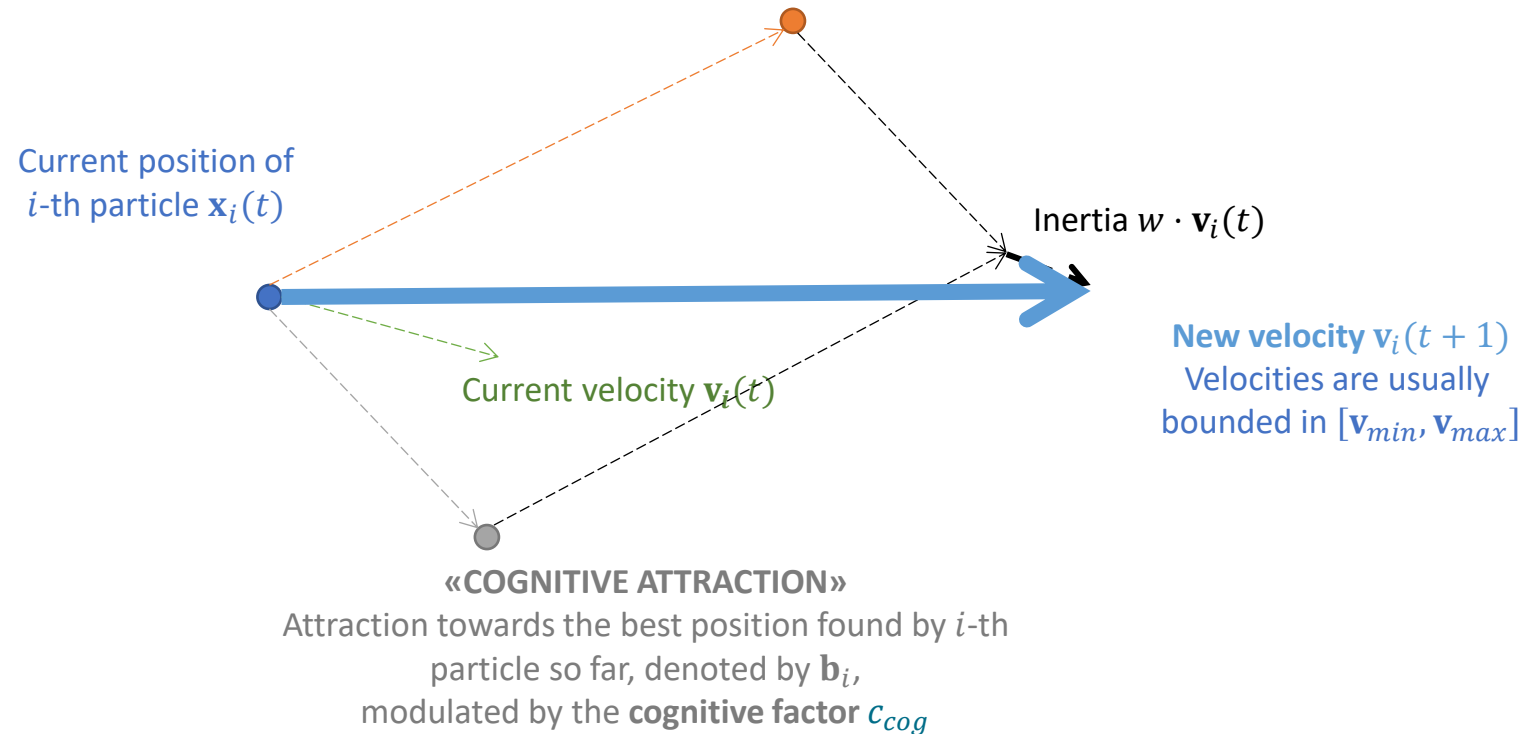
Particle Swarm Optimization [3/3]



A **swarm** of particles moving in the search space...

«SOCIAL ATTRACTION»

Attraction towards the **best position** found by the swarm so far, denoted by **g** , modulated by the **social factor** c_{soc}



Particle Swarm Optimization [3/3]



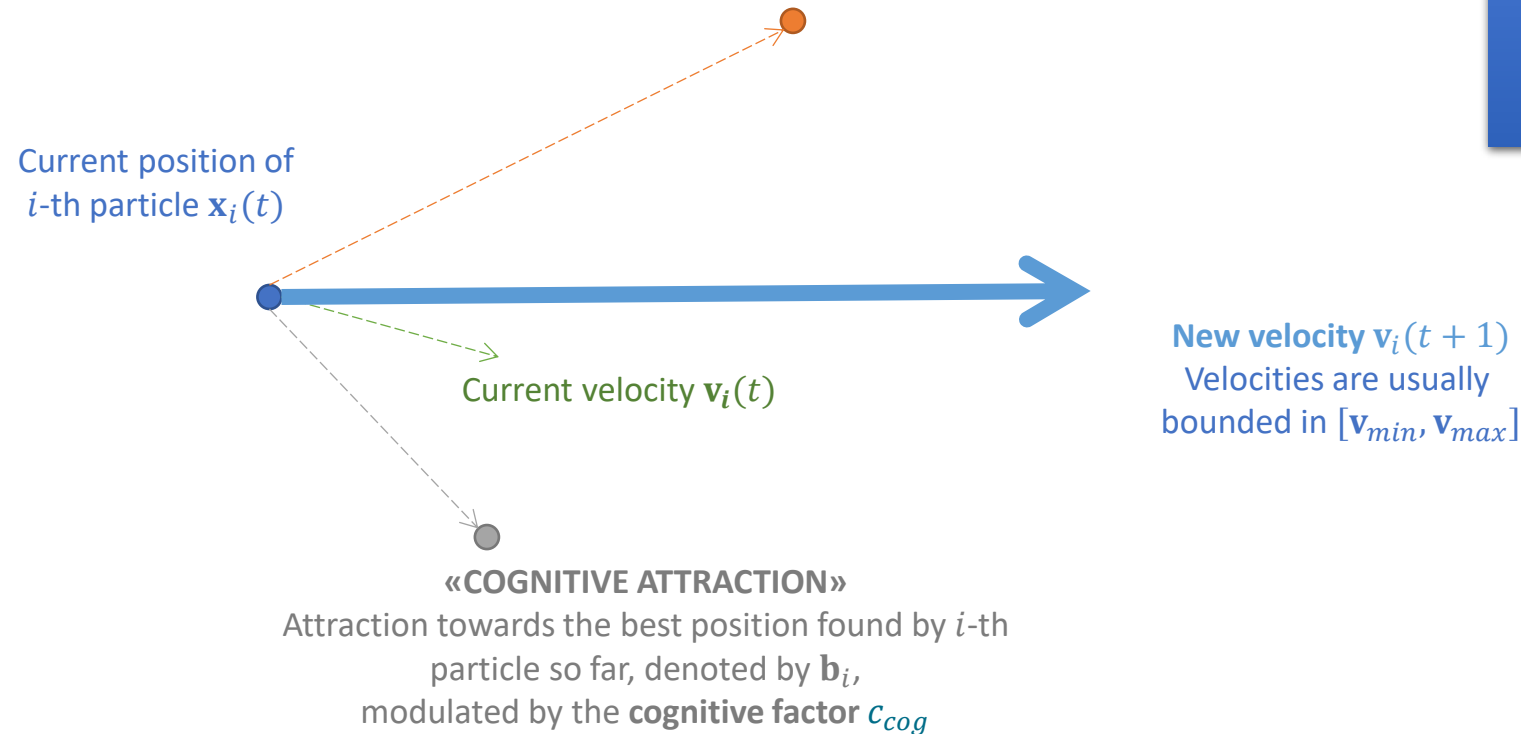
A **swarm** of particles moving in the search space...

«SOCIAL ATTRACTION»

Attraction towards the **best position** found by the swarm so far, denoted by **g**, modulated by the **social factor** c_{soc}

As soon as all positions are updated, the **fitness values** of all particles are updated

This information is used to **update** (if necessary) both the **b** and **g** vectors



Boundary conditions



During their «flight», particles could move **outside the search space**

- Those regions correspond to **unfeasible solutions**
- We could just **ignore those particles**: they would become «invisible» for the rest of the swarm, but we would lose exploration capability
- We could assign a **bad fitness value** (hopefully they will eventually return to the feasible region)

Or we can use **boundary conditions**!

- Move back the particle **ON** the boundary (absorbing): particles lose momentum
- Particles could «bounce» back (reflecting): better results than reflection
- **Damping** (i.e., stochastic «bounce») yields the best results
- [Xu *et al.*, IEEE Tran Antennas Propag 2007]

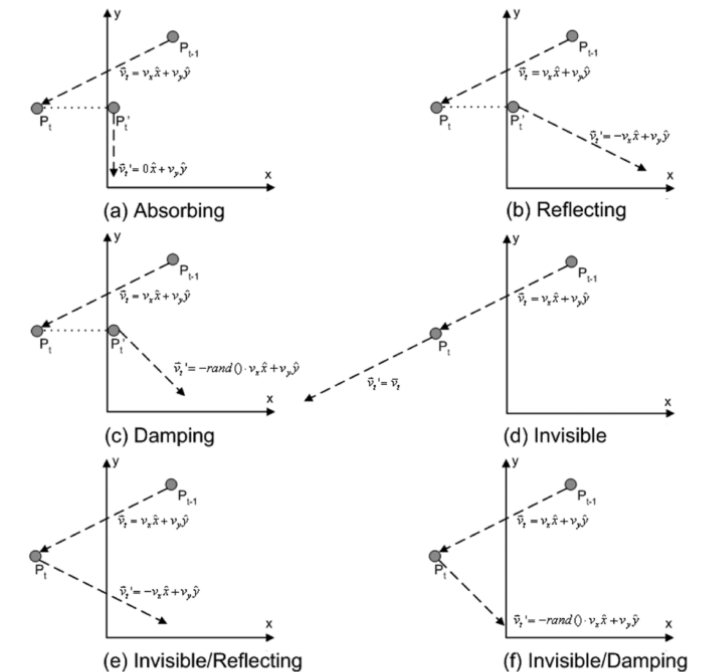


Fig. 3. Six different boundary conditions for a two-dimensional problem. P' and $\vec{v}'$ represent the modified position and velocity, respectively, after the errant particle is treated by boundary conditions.

Controlling the swarm: PSO's hyper-parameters



- PSO users must choose **multiple settings**:
 - The **social factor** $c_{soc} \in \mathbb{R}^+$, which modulates the attraction towards the «global best» («local exploitation» of the search space). Traditionally, $c_{soc} = 1.49445$
 - The **cognitive factor** $c_{cog} \in \mathbb{R}^+$, which modulates the attraction towards the «personal best» («global exploration» of the search space). Traditionally, $c_{cog} = 1.49445$
 - The **inertia weight** $w \in \mathbb{R}^+$, which balances the aforementioned settings: low inertia helps the local search, high inertia helps global search. Traditionally, w linearly decrements from 0.9 to 0.4
 - The **swarm size** N
 - (If not using the constriction factor) the **maximum velocity** of particles $\mathbf{v}_{\max} \in \mathbb{R}^M$
 - Possibly, also the **minimum velocity** of particles $\mathbf{v}_{\min} \in \mathbb{R}^M$ (do not lose momentum!) [Abraham and Liu, 2009]
 - Moreover: a **boundary condition strategy** must be selected, along with a **communication topology**
- All these hyper-parameters should be carefully chosen, since they have a **relevant impact** on the optimization: a bit annoying for people without expertise

Selection of PSO settings

The optimal settings for PSO are **problem-dependent**

- Problem of **meta-optimization**: optimize the settings of the optimizer
- **Issues** related to poor choice: divergence, cyclic trajectories, premature convergence, loss of momentum...
- Possible approach 1: **differential investigation** of the impact of the settings
- Possible approach 2: use **heuristic values** which are «good enough» in any situation (99% of papers)
- Possible approach 3: implement a **self-tuning** PSO, in which the **parameters are automatically adjusted** by means of some rules or algorithm

For instance, one could be interested in implementing rules like:

«when a particle gets too close to the best individual of the swarm, increase its cognitive factor»



Fuzzy PSO



- PSO has been **hybridized with Fuzzy Inference Systems** for the automatic selection of functioning settings **at run-time**

Authors	Settings calculated	Input variables
Shi and Eberhart (2001)	Inertia w	1) current best performance evaluation 2) current inertia value
Abraham and Liu (2009)	Minimum velocity $\mathbf{V}_{\min}$	1) current best performance evaluation 2) current velocity of particles
Tian and Li (2009)	Inertia w , «learning coefficient»	1) improvement of the global best 2) standard deviation of particles' fitness
Olivas <i>et al.</i> (2015)	c_{cog} , c_{soc}	1) diversity measure (based on distance) 2) error measure (diversity based on fitness)
Nobile <i>et al.</i> (2015)	w_i , c_{cog_i} , c_{soc_i}	1) difference in fitness value 2) distance from $\mathbf{g}$

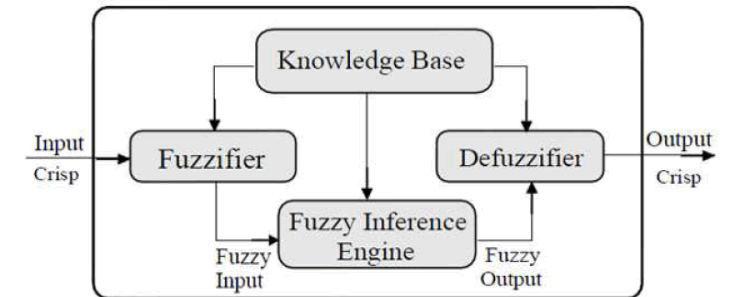


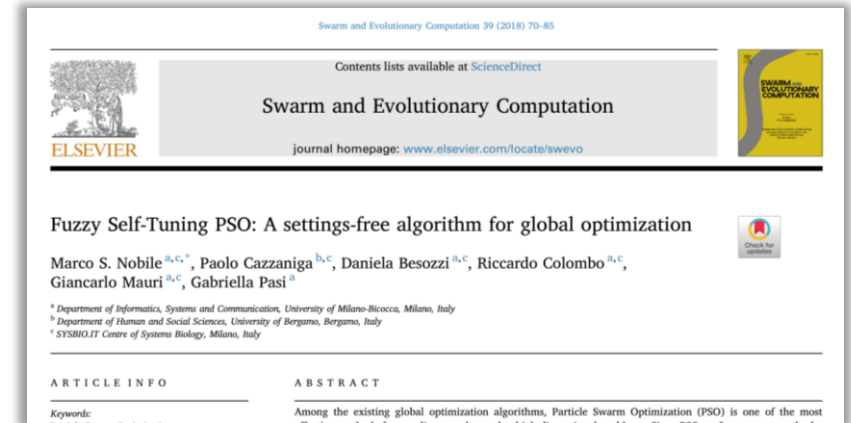
Fig.1. Block diagram of a fuzzy inference system

Fuzzy Self-Tuning PSO



FST-PSO is a **settings-free** meta-heuristic based on PSO

- Relies on Fuzzy Logic to give particles a «glimpse» of **additional intelligence** (more rules, a bit of introspection)
 - **No hyper-parameters**: completely automatic
 - FST-PSO calculates **inertia, social factor, cognitive factor, minimum velocity, and maximum velocity** at run-time
-
- Differently from previous fuzzy versions of PSO, FST-PSO sets all these values for **each particle** according to **its current performance** and **its distance from g**
 - [Nobile *et al.*, Swarm Evol Comp 2018]



Functioning of FST-PSO

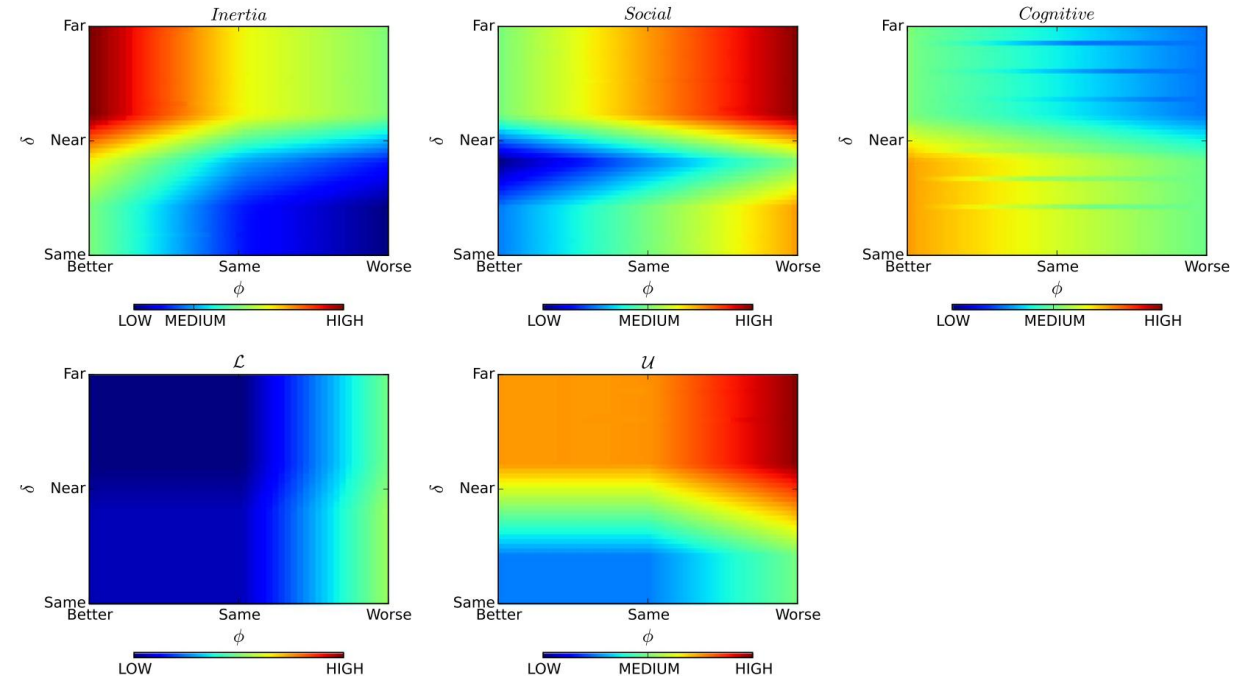


Each particle dynamically adapts **its own** hyper-parameters according to two information:

- The normalized **fitness improvement** with respect to the previous iteration ϕ
- The **distance from the global best** δ

Fuzzy rules used by FST-PSO.

Rule no.	Rule definition
1	if (ϕ is <i>Worse</i> or δ is <i>Same</i>) then (<i>Inertia</i> is <i>Low</i>)
2	if (ϕ is <i>Same</i> or δ is <i>Near</i>) then (<i>Inertia</i> is <i>Medium</i>)
3	if (ϕ is <i>Better</i> or δ is <i>Far</i>) then (<i>Inertia</i> is <i>High</i>)
4	if (ϕ is <i>Better</i> or δ is <i>Near</i>) then (<i>Social</i> is <i>Low</i>)
5	if (ϕ is <i>Same</i> or δ is <i>Same</i>) then (<i>Social</i> is <i>Medium</i>)
6	if (ϕ is <i>Worse</i> or δ is <i>Far</i>) then (<i>Social</i> is <i>High</i>)
7	if (δ is <i>Far</i>) then (<i>Cognitive</i> is <i>Low</i>)
8	if (ϕ is <i>Worse</i> or ϕ is <i>Same</i> or δ is <i>Same</i> or δ is <i>Near</i>) then (<i>Cognitive</i> is <i>Medium</i>)
9	if (ϕ is <i>Better</i>) then (<i>Cognitive</i> is <i>High</i>)
10	if (ϕ is <i>Same</i> or ϕ is <i>Better</i> or δ is <i>Far</i>) then ($\mathcal{L}$ is <i>Low</i>)
11	if (δ is <i>Same</i> or δ is <i>Near</i>) then ($\mathcal{L}$ is <i>Medium</i>)
12	if (ϕ is <i>Worse</i>) then ($\mathcal{L}$ is <i>High</i>)
13	if (δ is <i>Same</i>) then ($\mathcal{U}$ is <i>Low</i>)
14	if (ϕ is <i>Same</i> or ϕ is <i>Better</i> or δ is <i>Near</i>) then ($\mathcal{U}$ is <i>Medium</i>)
15	if (ϕ is <i>Worse</i> or δ is <i>Far</i>) then ($\mathcal{U}$ is <i>High</i>)



Effects of rules to particles according to δ and ϕ (calculated with Takagi-Sugeno inference engine)



Using FST-PSO

- FST-PSO is implemented in Python and can be easily installed with pip:
 - `pip install fst-pso`
- FST-PSO is settings-free. The user must provide only two information:
 1. The **search space** defined as a $2 \times D$ list or numpy array, containing with the minimum and maximum values for each coordinate
 2. The **fitness function**, defined as a python function taking as input a particle and returning the corresponding fitness value

When and why using PSO and its variants



- Well suitable for **real-valued problems** (although discrete versions exist)
- Simple to implement
- Effective in many real-world problems
- Does not require the problem to be differentiable (e.g., does not leverage the gradient)
- PSO is intrinsically parallel, e.g., can be accelerated on GPUs (see, e.g., [Nobile *et al.*, Proc EvoBIO 2012] or [Cagnoni *et al.*, Proc EvoAPPS 2012])



Examples of applications for PSO

- Optimal control, positioning, and design
- Neural network weights optimization
- Calibration of complex models (e.g., biochemical and biomedical models)
- Clustering and classification
- VLSI design, RF circuit synthesis, filter design
- Optimal energy dispatching, congestion management
- Decision making, investment portfolio selection
- Image analysis
- Prediction and forecasting
- See [Poli, Journal of Artificial Evolution and Applications 2008]

Stigmergy



Stigmergy is a mechanism of **indirect coordination** between agents **mediated by the environment**

- Stigmergy is a form of **self-organization**: it produces complex, seemingly intelligent structures, without the need for any planning, control, or even direct communication between the agents
- Stigmergy supports efficient collaboration between extremely **simple agents**, who lack any memory, intelligence or even individual awareness of each other (see [Heylighen, Cogn Syst Res 2016])

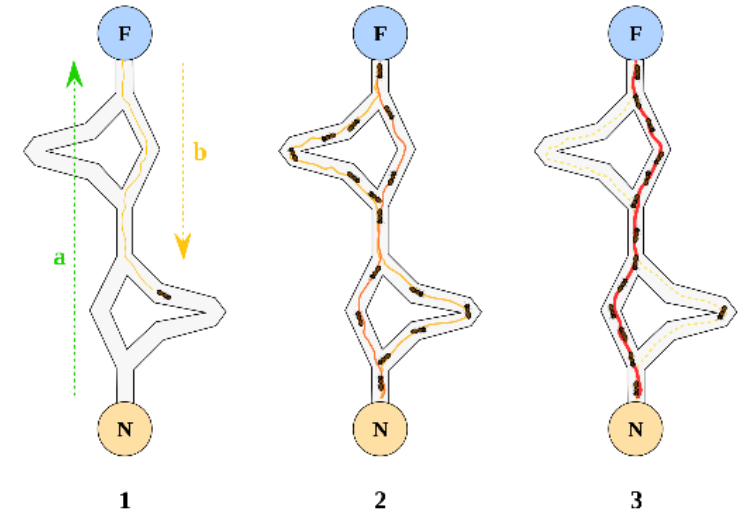
A **stigmergic system** displays three characteristics:

1. **A global environment** (comprised of multiple local environments) **partially perceivable** through an internal dynamics that governs its temporal evolution
 2. **A set of agents** populating the environment with **no single agent** (nor cluster of agents) **having global knowledge** of the system's state. Rationality is bounded, behavior self-organizes and emerges (through adaptation and stochasticity)
 3. **Novel features arise** from interactions of 1 and 2, features that are neither predictable nor reducible to simpler constituents (complexity)
- See [Marsh and Onof, Cogn Syst Res 2007]

Example of stigmergy: ants' pheromone trails

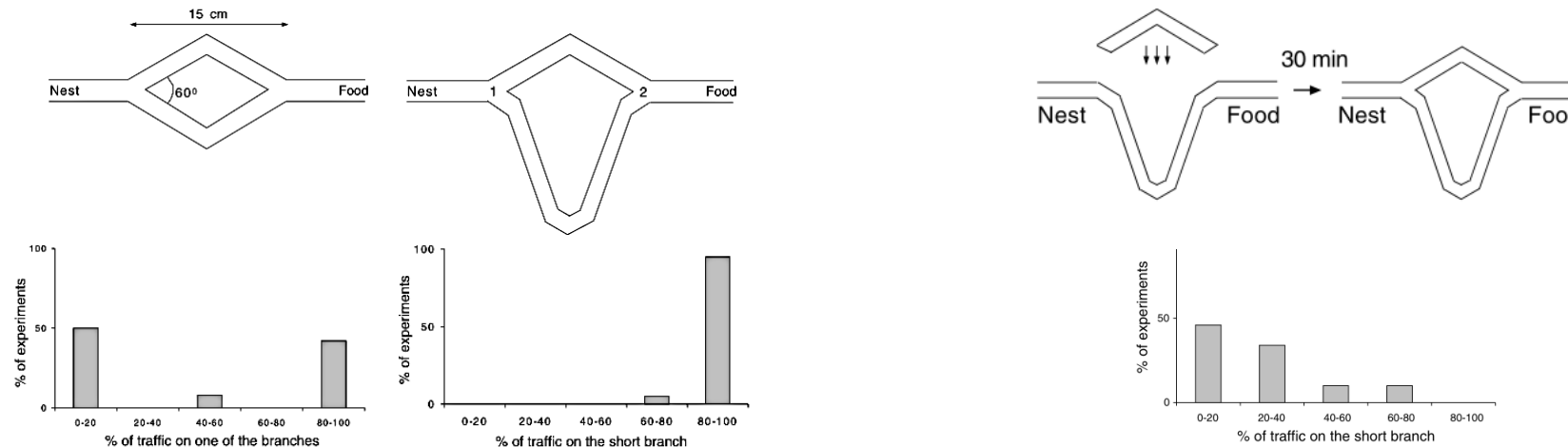


- The figure sketches the mechanisms of **stigmergy** (i.e., environment-mediated communication)
 - An ant leaves the nest **N** and wanders **randomly**
 - During the walk, **the ant leaves pheromone** on the ground
 - Eventually, the ant reaches the food source **F**
 - The ant can now **go back to N** leaving further pheromone
- Ants taking shorter paths can move from N to F and back in **less time**
 - Hence, they leave **more pheromone** on the ground
 - As a result, following ants will feel more pheromone on the **shortest path** between N and F (red line)
 - Pheromone trails are “**messages**” for the rest of the nestmates



Double bridge experiments

- Lab experiment: ants must find the **shortest route** between nest and food and back
 - Shorter paths tend to be **reinforced** with more pheromone
 - More reinforced paths are **more attractive for ants**; pheromone trails on longer paths **evaporate**
 - If we remove the shortest bridge, **the colony adapts** to the new environment by using the longest bridge (ants can adapt to dynamic environments!)
 - If the path is re-attached after 30', **due to stigmergy** the ants **keep using the longer route**
- See [Goss *et al.*, Naturwissenschaften 1999]



Ant Colony Optimization



Ant Colony Optimization (ACO)

- Meta-heuristic inspired by the collective **behavior of ants**
- Designed for **combinatorial optimization** (applied to TSP, VRP, graph coloring, knapsack problems, planning, scheduling, circuit board design...)
- [Dorigo *et al.*, Fut Gen Comp Syst 2000]

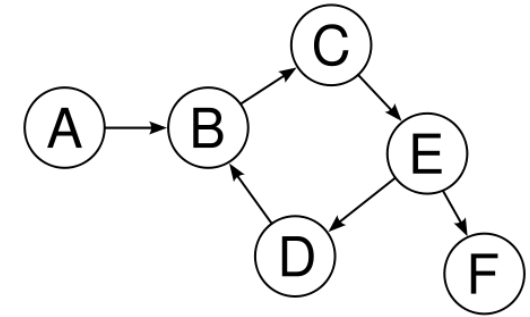
ACO relies on **stigmergy** and **simulated pheromone trails**

- Pheromone is **left by ants** in the search space and **evaporates over time**
- Pheromone traces **guide the behavior** of the ants
- Ants are **stochastic solution building procedures** exploiting both simulated pheromone and available **heuristic information** about the problem being solved

Paths and graphs



- A **graph** is a mathematical abstract structure representing **relationships** between pairs of «objects»
 - Objects are represented by **nodes** or vertices
 - Relationships are represented by **edges** connecting the nodes
 - In directed graphs, the edges have an **orientation** and are named **arcs**
- Back to the biological metaphor, the **possible paths** between nest and food can be formalized using a graph $G = (V, A)$
 - V is a set of nodes, while A is the set of arcs connecting the nodes, representing **possible paths**
 - Each arc (i, j) has got a **heuristic information** $\eta_{i,j}$ and a **pheromone** trail $\tau_{i,j}$





ACO algorithm

- The ACO algorithm is pretty simple:

```
while not termination_criterion():  
    build_ant_solutions()  
    update_pheromones()  
    daemon_actions() # optional  
return best solution
```

ACO algorithm

- The ACO algorithm is pretty simple:

```
while not termination_criterion():  
    build_ant_solutions()  
    update_pheromones()  
    daemon_actions() # optional  
return best solution
```

During this phase, **each ant builds a candidate solution**

For instance, **a random path on the graph G**

All solutions are evaluated using the **fitness function**

ACO algorithm

- The ACO algorithm is pretty simple:

```
while not termination_criterion():  
    build_ant_solutions()  
    update_pheromones()  
    daemon_actions() # optional  
return best solution
```

During this phase, the pheromone trails are **updated** in the environment

Already existing **pheromone** traces slowly **evaporate**

New pheromone is deposited by ants according to the **visited paths**



ACO algorithm

- The ACO algorithm is pretty simple:

```
while not termination_criterion():  
    build_ant_solutions()  
    update_pheromones()  
    daemon_actions() # optional  
return best solution
```

All **non-stigmergic actions** (i.e., centralized actions) are performed during this phase

For instance, **local search** steps to **fix/improve** the candidate solutions

Can be used to implement **elitism**

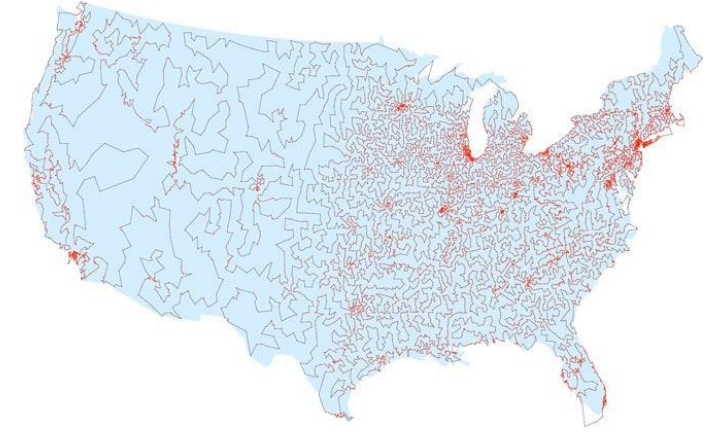
Building candidate solutions



- The first step of the algorithm is to “**simulate**” the ants, that is, to **build candidate solutions**
- The way solutions are built **depends on the problem** under investigation
- Regardless the actual algorithm used for building candidate solutions, ACO generally uses a **bottom-up approach**: it begins from a small “piece” of solution and keep adding pieces (=ant’s walk)
 - For instance, if the solution is a path in a graph (e.g., TSP), the construction process begins by selecting a random node
 - Then, the ant iteratively adds new arcs to the solution



Simple applied example of ACO



The shortest traveling salesman route going through all 13,509 cities in the United States with a population of at least 500 (as of 1998)
Image courtesy of D. Applegate, R. Bixby, V. Vovk and W. Cook

- **Traveling Salesman Problem (TSP)**
 - Find the minimum Hamiltonian circuit
 - **NP-complete problem** [Garey and Johnson, 1979]
- In ACO terms
 - Cities == Nodes (i.e., ants build paths between cities)
 - *Desiderability* of visiting city j after city i == Pheromone trails on arc (i, j)
 - The **heuristic** information is the **distance** between cities, i.e., $\eta_{i,j} = \frac{1}{d(i,j)}$ where $d(i,j)$ is the distance between the cities i and j
 - The **goal** is to find a solution (i.e., path) that **minimizes the sum of the distances**
 - We denote $\mathcal{N}_i^k$ as the valid **neighborhood** of city i for ant k (i.e., cities that are connected to i and were not added to i , yet)

Ants and path building

- In classic ACO for TSP, the probability for ant k of selecting a city j after city i is calculated as

$$p_{i,j}^k(t) = \frac{[\tau_{i,j}(t)]^\alpha \cdot [\eta_{i,j}]^\beta}{\sum_{l \in \mathcal{N}_i^k} [\tau_{i,l}(t)]^\alpha \cdot [\eta_{i,l}]^\beta}$$

provided that $j \in \mathcal{N}_i^k$

Ants and path building

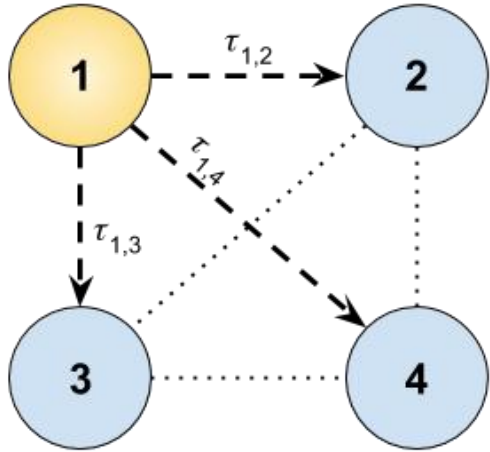
- In classic ACO for TSP, the probability for ant k of selecting a city j after city i is calculated as

$$p_{i,j}^k(t) = \frac{[\tau_{i,j}(t)]^\alpha \cdot [\eta_{i,j}]^\beta}{\sum_{l \in \mathcal{N}_i^k} [\tau_{i,l}(t)]^\alpha \cdot [\eta_{i,l}]^\beta}$$

provided that $j \in \mathcal{N}_i^k$

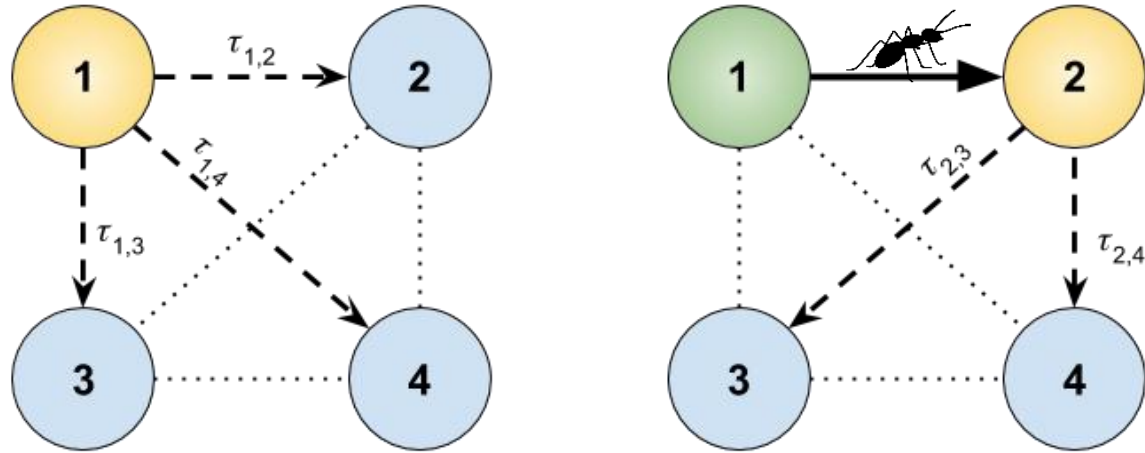
- Please note that:
 - $\alpha = 0$ implies that **pheromone traces are neglected**, the **heuristic information dominates** and the algorithm becomes a **greedy search**
 - $\beta = 0$ implies that the **heuristic information is discarded**, and the construction of solutions is **exclusively based on the pheromone traces**

Example of solution construction only considering pheromone



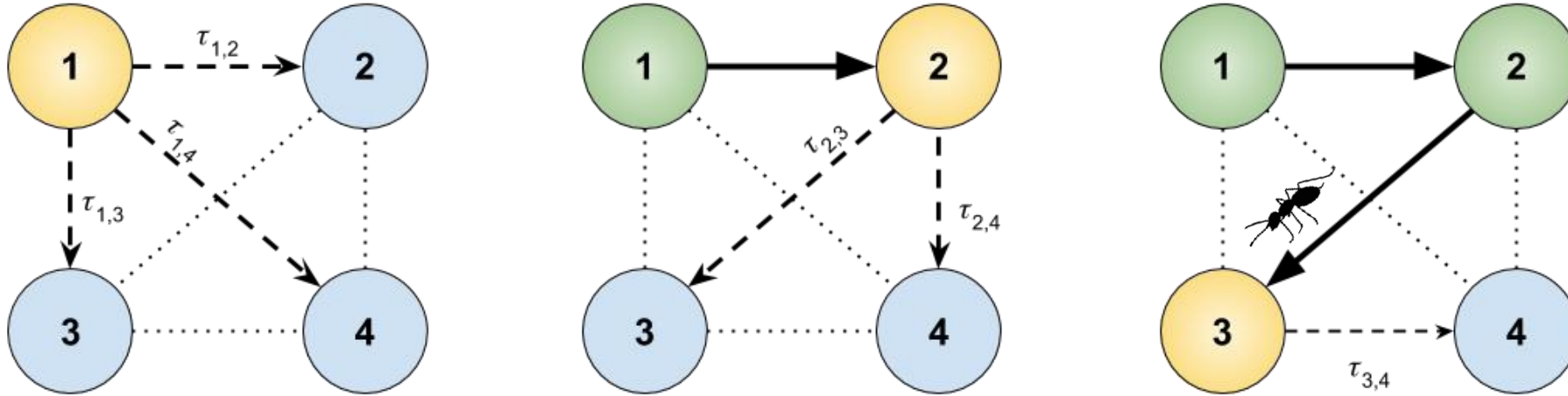
- Example of solution built by ant k in the case of a 4-cities TSP instance
 - Assume (for simplicity) that all distances between cities are equal to 1
 - A random node (1) is chosen
 - $\mathcal{N}_1^k = \{2,3,4\}$
 - $$p_{1,j}^k(t) = \frac{\tau_{1,j}}{\tau_{1,2} + \tau_{1,3} + \tau_{1,4}}$$

Example of solution construction only considering pheromone



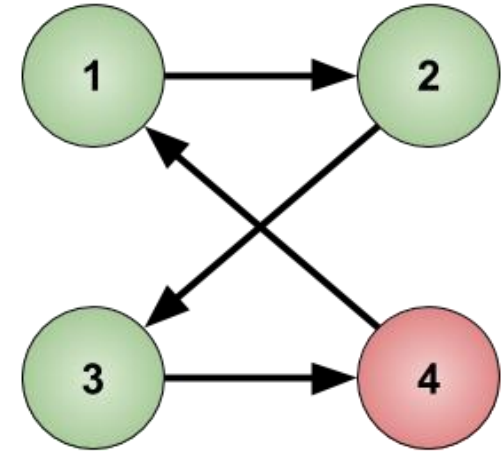
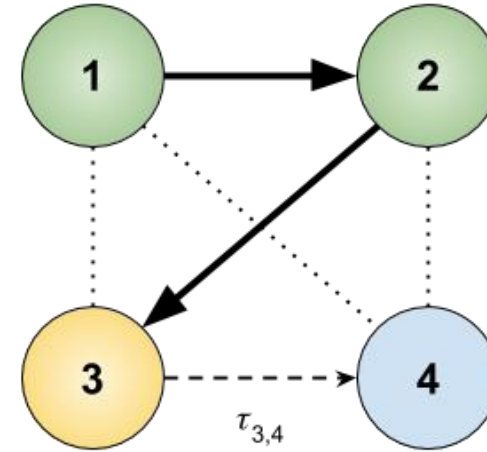
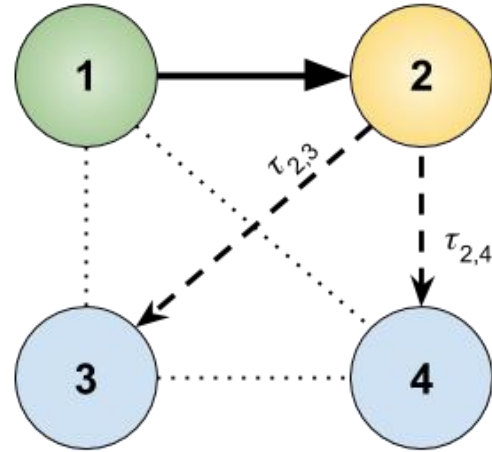
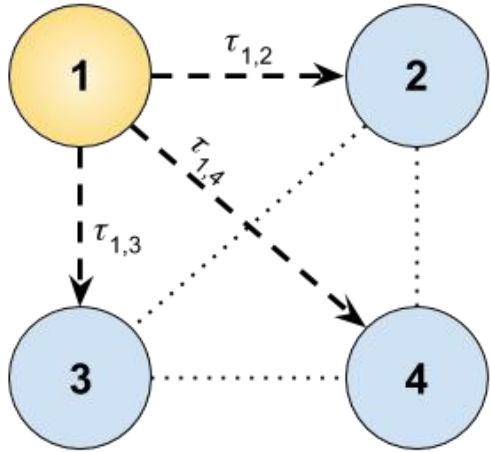
- One edge is chosen according to the calculated probabilities
 - In this case, we chose the edge (1,2)
 - The partial solution so far is the sequence of cities 1-2
 - $\mathcal{N}_2^k = \{3,4\}$
 - $$p_{2,j}^k(t) = \frac{\tau_{1,j}}{\tau_{2,3} + \tau_{2,4}}$$

Example of solution construction only considering pheromone



- One edge is chosen according to the calculated probabilities
 - In this case, we chose the edge (2,3)
 - The partial solution so far is the sequence of cities 1-2-3
 - $\mathcal{N}_2^k = \{4\}$
 - $p_{3,4}^k(t) = 1$
 - The algorithm has no further choices, so it picks node 4 and closes the loop

Example of solution construction only considering pheromone



- The final candidate solution is 1-2-3-4
 - The fitness value of the final solution is $d(1,2) + d(2,3) + d(3,4) + d(4,1)$



What changes with the heuristic information?

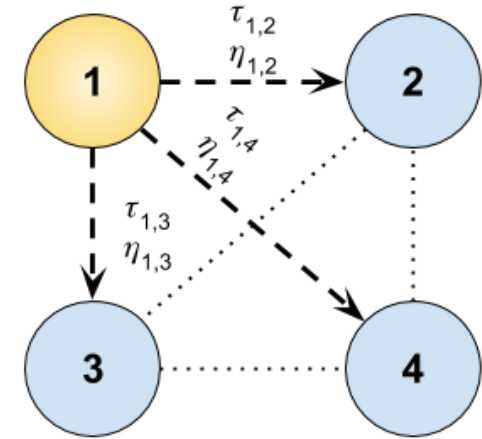


- If we consider the heuristic information, the probability distribution changes

$$p_{1,j}^k(t) = \frac{\tau_{1,j} \eta_{1,j}}{\tau_{1,2} \eta_{1,2} + \tau_{1,3} \eta_{1,3} + \tau_{1,4} \eta_{1,4}}$$

- The probability is now biased by the heuristic information
 - That is, edges with same pheromone but **larger heuristic value** have a **higher probability** of being selected for solution construction (remember that, for TSP, $\eta_{i,j} = \frac{1}{d(i,j)}$)
 - Please note that pheromone and heuristic information have different units and magnitudes: we can **balance the two influences by means of the hyper-parameters α and β**

$$p_{i,j}^k(t) = \frac{[\tau_{i,j}(t)]^\alpha \cdot [\eta_{i,j}]^\beta}{\sum_{l \in \mathcal{N}_i^k} [\tau_{i,l}(t)]^\alpha \cdot [\eta_{i,l}]^\beta}$$



Pheromone update

- Positive feedback: the **better solutions** are used to **increase the pheromone** traces
 - Or **just the best solution** found in the case of «MAXMIN ACO»
 - Existing pheromone slowly **evaporates**: prevents saturation and helps to «forget» bad paths
 - **Pheromone evaporation** is controlled with an additional hyper-parameter $0 < \rho < 1$
- In classic «Ant Systems» the **pheromone update formula** is

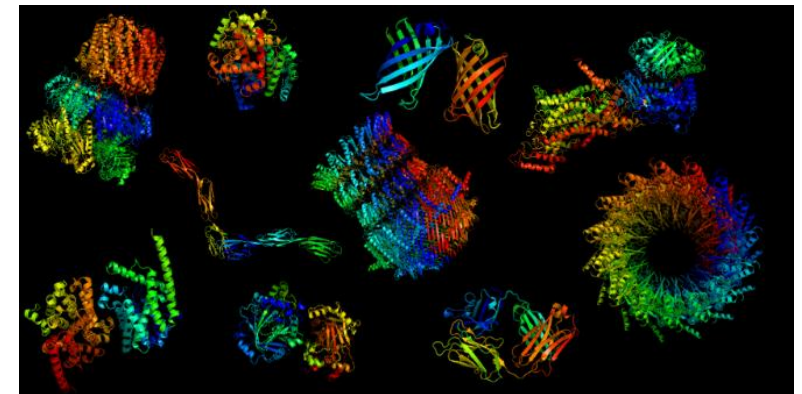
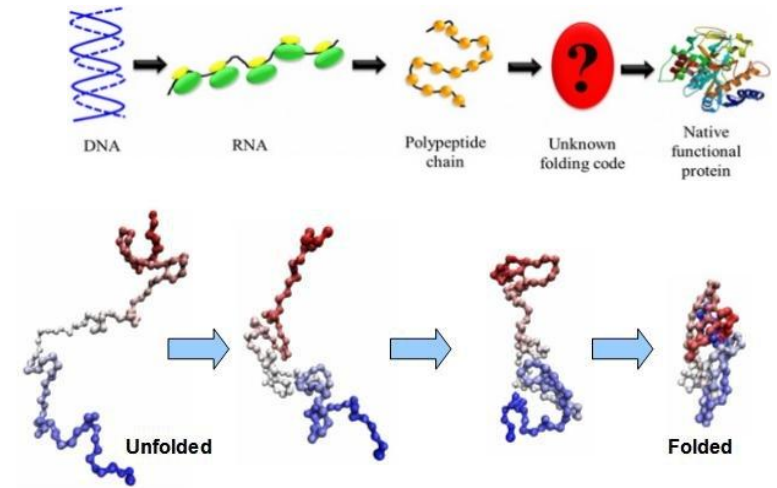
$$\tau_{i,j}(t) = \overset{\text{Evaporation}}{(1 - \rho) \cdot \tau_{i,j}(t - 1)} + \overset{\text{Deposit}}{\sum_{k=1}^N \Delta\tau_{i,j}^k} \quad \forall (i, j) \in E$$

- where N is the number of ants in the colony and $\Delta\tau_{i,j}^k = \frac{1}{L_k}$
 L_k is the length of ant k 's path (provided that arc (i, j) belongs to ant k 's solution)

Applied ACO: the HP problem [1/3]

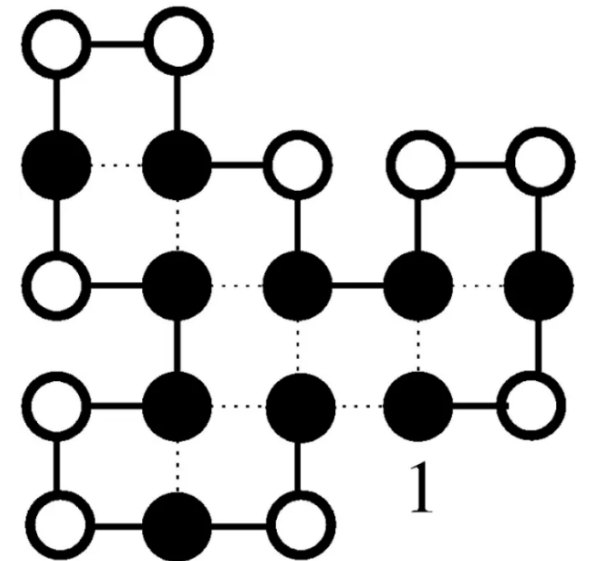


- One of the most challenging open problems in computational biology is the ***ab-initio* protein folding**
- That is, the identification of **lowest-energy** conformations starting from amino-acid sequences
- **Structure == function, misfolding == onset of diseases**



Applied ACO: the HP problem [2/3]

- Hydrophobic-Polar protein folding problem (HP)
 - Simplified lattice-based model in which the sequence of amino-acids is abstracted by using strings of hydrophobic (H •) and polar (P ○) residues
 - Key assumptions of HP model: **self-avoidance** (no overlap of amino-acids), the **sequence cannot be broken**, H-P contacts must be **avoided** (=hi energy), while H-H contacts are **avored** (=low energy)
 - **NP-hard** (minimization) problem
- **ACO can be used** to tackle this problem
 - [Shmygelska and Hoos, BMC Bioinform 2005]
 - Interesting example of non-trivial solution construction

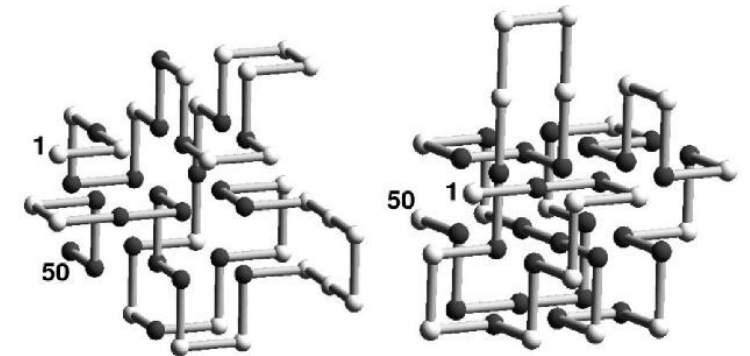
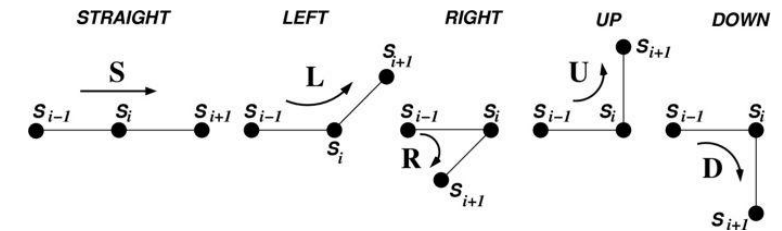


A sample protein conformation in the 2D HP model. The underlying protein sequence (Sequence S1-1 from Table 1) is HPHPPHHPHPPHPPHPPHPPH; black circles represent hydrophobic amino acids, while white circles symbolise polar amino acids. The dotted lines represent the H-H contacts underlying the energy calculation. The energy of this conformation is -9, which is optimal for the given sequence.

Applied ACO: the HP problem [3/3]



- The ACO builds the **path of the protein** in the 3D space using simple local structure motifs
 - Such motifs represent **changes in direction** from the *perspective* of the previous amino-acid
 - This representation is **rotationally invariant**, so both the position and orientation of the first amino-acid are not relevant
 - Such direction is chosen probabilistically by the ants according to pheromone and heuristic information
 - The **pheromone** encodes the previous **experience**
 - The **heuristic information** encodes the (current) **energy state**
 - The algorithm is memetic, i.e., it also performs a **local search** phase, using **long-range mutations** to further reduce the protein's energy and avoid local minima



When to use ACO?



- Excellent choice for **constrained discrete problems**
- Can be used in **dynamic applications** (e.g., it adapts to new distances in TSP)
- When you have some optimization problem over **a graph**

Eusociality

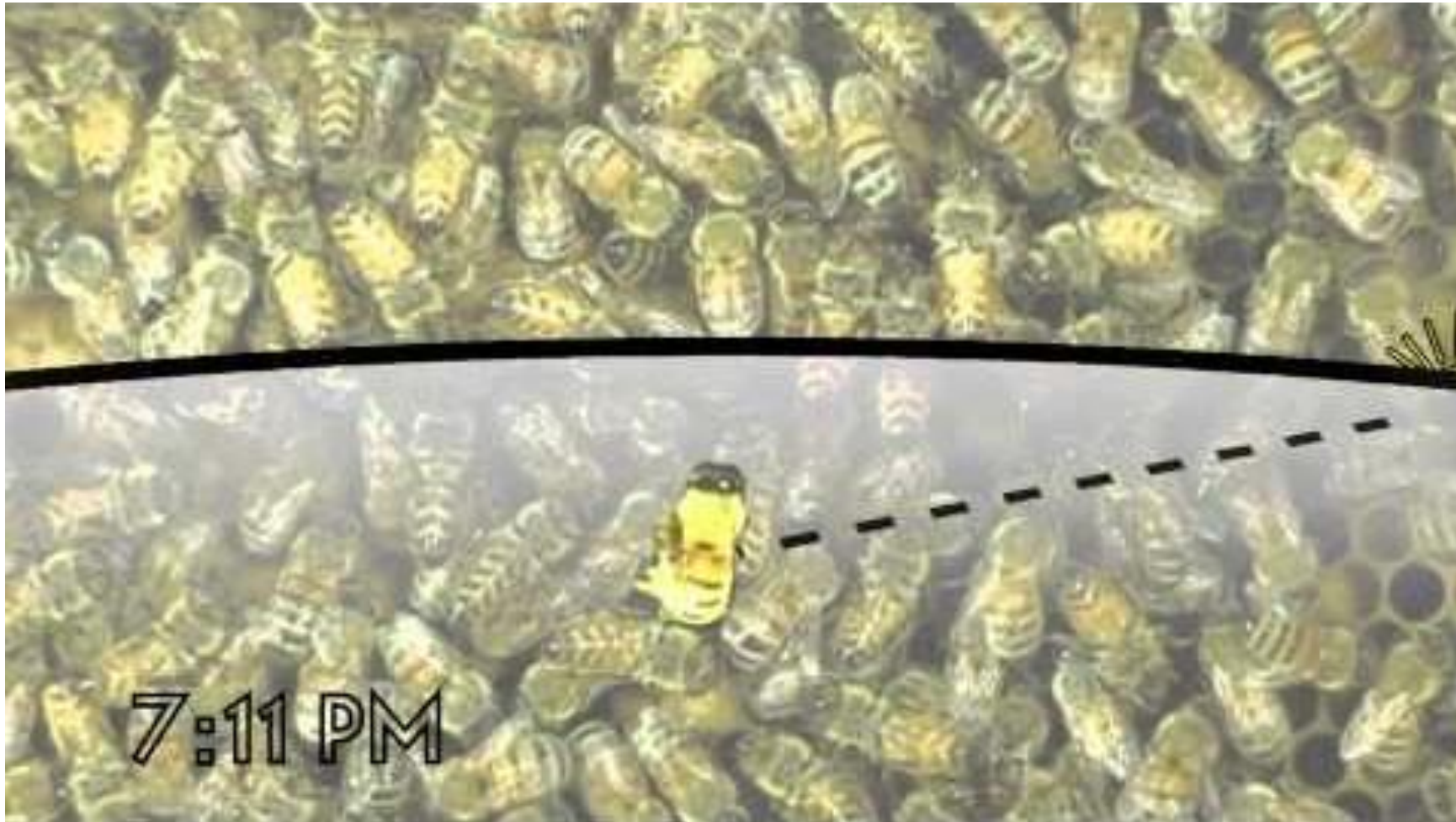


Highest level of **social organization** in colonies of animals

- Eusocial behavior is associated with several traits and, in particular, with an explicit **division of social roles**, marked by intracolony reproductive hierarchies
- Many insects belonging to the order of **Hymenoptera** are eusocial (nearly all ants, some bees, and wasps)
- Social roles are distributed into **castes** that include the reproductive queen(s), female (sterile) **workers which forage for pollen** and care for the brood and nest, and brood composed of potential **workers** and reproductive males and gyne
- Bees can use **pheromones** to help foraging: workers bees of *Tetragonula carbonaria*, for instance, mark food sources with a pheromone, assisting nestmates to find the food. However, **this is generally not the case**



Artificial Bee Colony: the inspiration



Artificial Bee Colony



Artificial Bee Colony (ABC) is a Swarm Intelligence meta-heuristic for optimization

- (Loosely) inspired by the **foraging behavior of honey bees**
- «Food» and «nectar» are the metaphors for candidate solutions and fitness values, respectively

In ABC, bees can have three roles: employed, onlookers, and scouts

- **Employed** are assigned to food sources around the hive
- **Onlookers** are distributed around the food sources, in **proportion** to the nectar amount
- **Scouts** perform **random search** in the search space, looking for new food sources
- [Karaboğa and Basturk, J Glob Optim 2007]



General functioning of the ABC algorithm



The idea is that the employed bee finds a food source and goes back to the nest to do the waggle dance

- Each **onlooker** bee watches the waggle dances and **chooses** (probabilistically) one food source to visit and «exploit»
- The probability of visiting a food source is **proportional to the nectar** (→ fitness value)
- Onlookers «exploit» the food sources by using some **user-defined neighborhood function** (e.g., they randomly perturb the coordinates of food sources to evaluate new candidate solutions)
- **Employed** bees whose solutions cannot be improved for a predetermined number of **trials** (sometimes called the «limit») leave the food source and **become scouts**
- There are some hidden hyper-parameters in the ABC algorithm: the proportion of employed, onlooker, and scout bees, the parameters governing the neighborhood function, the number of trials

Summary



- Swarm Intelligence takes inspiration from the emergent intelligence of groups of animals (e.g., ants, bees, birds)
- It is a sort of agent-based simulation, whose goal is to minimize a function
- It can be effective to tackle optimization problems (either continuous or even combinatorial)
- Fuzzy logic can be used to "extend" the behavior of the agents and take more and more autonomous decisions

Velocity and constriction factor

- The **final equation for the velocity update** is the following:

$$\mathbf{v}_i(t+1) = w \cdot \mathbf{v}_i(t) + c_{soc} \cdot \mathbf{r}_1 \otimes (\mathbf{g} - \mathbf{x}_i(t)) + c_{cog} \cdot \mathbf{r}_2 \otimes (\mathbf{b}_i - \mathbf{x}_i(t))$$

where $\mathbf{r}_1$ and $\mathbf{r}_2$ are vectors or random numbers sampled with uniform distribution in $[0,1]$ and $\otimes$ denotes the Hadamard product

- Velocity is usually clamped** to prevent a chaotic movement of particles; as an alternative, a variant using a **constriction factor** was proposed:

$$\mathbf{v}_i(t+1) = \chi(\mathbf{v}_i(t) + c_{soc} \cdot \mathbf{r}_1 \otimes (\mathbf{g} - \mathbf{x}_i(t)) + c_{cog} \cdot \mathbf{r}_2 \otimes (\mathbf{b}_i - \mathbf{x}_i(t)))$$

$$\chi = \frac{2k}{|2 - \varphi - \sqrt{\varphi^2 - 4\varphi}|}$$

where $k \in [0,1]$, $\varphi = c_{soc} + c_{cog}$ and $\varphi > 4$



Restarts

- Classic PSO is based on the movement of agents and does not employ any explicit mutation mechanism/operator
- **PSO can actually be extended with mutation**, and the best mutation strategy is **problem-dependent** [Andrews, CEC 2006]
- A different approach could be the **restart** (or reboot): **recreate one or more particles** when **premature convergence** (or **stagnation**) is detected
 - Given the **same budget of fitness evaluations**, restarts can improve final performances
 - See [Spolaor *et al.*, CIBCB 2017] for some restart strategies applied to PSO

Elitism and exploration

- We can implement **elitism** by forcing ACO to «remember» the best solution meet so far, by adding a further term:
- $\tau_{i,j}(t) = (1 - \rho) \cdot \tau_{i,j}(t - 1) + \sum_{k=1}^N \Delta\tau_{i,j}^k + e \cdot \Delta\tau_{i,j}^{best}$
- We can also do the opposite! The ants «**consume**» **pheromone** when they visit a position (see Ant Colony Systems), this strategy promotes **explorative behavior**
- $\tau_{i,j}(t) = (1 - \xi) \cdot \tau_{i,j}(t - 1) + \xi \cdot \tau_0$
 - where $\xi \in (0,1]$ and τ_0 is the initial value of pheromone